

8086 Microprocessor: Questions and Answers

Q1: What are the main functional units of 8086?

Answer: The 8086 has two main functional units:

- **Bus Interface Unit (BIU)** – Handles external bus operations like instruction fetching, memory access, and I/O.
- **Execution Unit (EU)** – Decodes and executes instructions using the ALU and registers.

Q2: What is the role of the Bus Interface Unit (BIU)?

Answer: The BIU is responsible for:

- Generating physical addresses using segment and offset registers.
- Managing the instruction queue.
- Controlling data flow between CPU, memory, and I/O devices.

Q3: What is the role of the Execution Unit (EU)?

Answer: The EU:

- Decodes and executes instructions.
- Uses general-purpose registers and the ALU for computations.
- Updates the flag register after every operation.

Q4: What is the purpose of the instruction queue in 8086?

Answer:

- The BIU prefetches up to 6 bytes of instructions into the queue.
- The EU takes instructions from the queue for execution.
- Enables **pipelining** – overlapping of fetch and execute cycles.

Q5: What is pipelining in the 8086 microprocessor?

Answer: **Pipelining** allows the BIU to fetch the next instruction while the EU executes the current one. This improves performance by overlapping operations and reducing idle CPU time.

Q6: Why does the 8086 have a 20-bit address bus if it is a 16-bit processor?

Answer: Because 8086 uses a **Segment:Offset** addressing scheme:

$$\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$$

This allows access to $2^{20} = 1 \text{ MB}$ of memory space even with 16-bit registers.

Q7: Give an example of physical address calculation in 8086.

Example:

$$\text{CS} = 348A_h, \quad \text{IP} = 4216_h$$

Step 1: $348A_h \times 10_h = 348A0_h$ **Step 2:** $348A0_h + 4216_h = 38C16_h$

Result:

Physical Address = $38C16_h$

Q8: How much memory can 8086 directly address?

Answer: With a 20-bit address bus:

$$2^{20} = 1,048,576 \text{ bytes} = 1 \text{ MB}$$

Hence, the 8086 can address **up to 1 MB of memory**.

Q9: What are the main categories of registers in 8086?

Answer: Registers in 8086 are classified into five main categories:

- General Purpose Registers (GPRs)
- Segment Registers
- Pointer and Index Registers
- Instruction Pointer (IP)
- Flag Register

Each category performs specific data storage and control tasks within the CPU.

Q10: What are General Purpose Registers and their uses?

Answer: The 8086 has four 16-bit General Purpose Registers:

- **AX (Accumulator)** – Used for arithmetic, logic, and I/O operations.
- **BX (Base)** – Acts as a base pointer for memory addressing.
- **CX (Count)** – Used as a loop counter and in shift/string operations.
- **DX (Data)** – Used in I/O and multiplication/division instructions.

Each register can be divided into two 8-bit parts (AH/AL, BH/BL, CH/CL, DH/DL).

Q11: What is the function of the flag register?

Answer: The flag register holds **status flags** that indicate the outcome of arithmetic or logical operations:

- Carry Flag (CF)
- Zero Flag (ZF)
- Sign Flag (SF)
- Overflow Flag (OF)
- Parity Flag (PF)

Q12: What are Segment Registers and why are they needed?

Answer: Segment registers are used to access memory in the segmented architecture of 8086.

- **CS** – Code Segment (program instructions)
- **DS** – Data Segment (variables and data)
- **SS** – Stack Segment (stack operations)
- **ES** – Extra Segment (string operations)

They help in generating 20-bit physical addresses using:

$$\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$$

Q13: What are Pointer and Index Registers used for?

Answer: These registers assist in stack management and string operations:

- **SP (Stack Pointer)** – Points to the top of the stack.
- **BP (Base Pointer)** – Accesses stack parameters.
- **SI (Source Index)** – Source for string operations.
- **DI (Destination Index)** – Destination for string operations.

Q14: What is the function of the Instruction Pointer (IP)?

Answer: The **Instruction Pointer (IP)** is a 16-bit register that:

- Holds the offset of the next instruction within the code segment.
- Works with CS to form the 20-bit physical address:

$$\text{Physical Address} = (\text{CS} \times 16) + \text{IP}$$

- Automatically increments after each instruction fetch.

Q15: What is the purpose of the Flag Register in 8086?

Answer: The 16-bit **Flag Register** reflects the CPU's current status and control states.

- **Status Flags:** CF, PF, AF, ZF, SF, OF – indicate arithmetic results.
- **Control Flags:** TF, IF, DF – manage CPU operations and interrupts.

Each flag is set or cleared depending on the outcome of operations.

Q16: How do registers support memory addressing in 8086?

Answer: 8086 combines a segment register with an offset register to produce a 20-bit address:

$$\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$$

This segmentation allows access to **1 MB of memory** using 16-bit registers.

Q17: Summarize the overall role of registers in 8086.

Answer: Registers form the backbone of the 8086 CPU by:

- Holding operands and intermediate results.
- Managing program control and stack operations.
- Supporting segmented memory addressing.
- Tracking processor status and control using flags.

Q18: What is the purpose of the Flag Register in 8086?

Answer: The **Flag Register** is a 16-bit special-purpose register that indicates the current status of the processor and controls certain operations. It consists of:

- **Status (Condition) Flags** – Show the result of arithmetic or logic operations.
- **Control Flags** – Affect the processor's operation and instruction execution.

Q19: How many active flags are present in the 8086 Flag Register?

Answer: The 8086 Flag Register contains **9 active flags**:

- **6 Condition Flags** – CF, PF, AF, ZF, SF, OF
- **3 Control Flags** – TF, IF, DF

Q20: What are Condition Flags in 8086?

Answer: Condition Flags represent the results of arithmetic or logical operations:

- **CF (Carry Flag)** – Set when an arithmetic operation generates a carry or borrow.
- **PF (Parity Flag)** – Set if the result has even parity (even number of 1's).
- **AF (Auxiliary Carry Flag)** – Set if a carry occurs from bit 3 to bit 4.
- **ZF (Zero Flag)** – Set if the result of an operation is zero.
- **SF (Sign Flag)** – Reflects the sign of the result (MSB = 1 → negative).
- **OF (Overflow Flag)** – Set when signed overflow occurs.

Q21: What is the function of the Carry Flag (CF)?

Answer: The **Carry Flag** indicates an overflow out of the most significant bit (for unsigned arithmetic).

- Set to 1 – if a carry (addition) or borrow (subtraction) occurs.
- Cleared to 0 – if no carry or borrow occurs.

Q22: What is the purpose of the Zero Flag (ZF)?

Answer: The **Zero Flag** is set when the result of an arithmetic or logical operation equals zero.

$$\text{If Result} = 0 \Rightarrow ZF = 1$$

Otherwise, $ZF = 0$. It is widely used in conditional jump and loop instructions.

Q23: Explain the Overflow Flag (OF) with its significance.

Answer: The **Overflow Flag** is used in signed arithmetic operations. It is set when the result of a signed operation exceeds the representable range of a signed number.

$$\text{Example: } (+127) + (+1) = -128 \Rightarrow OF = 1$$

Q24: What are Control Flags in 8086?

Answer: Control Flags control the operation of the processor:

- **TF (Trap Flag)** – Enables single-step execution for debugging.
- **IF (Interrupt Flag)** – Enables or disables maskable interrupts.
- **DF (Direction Flag)** – Controls the direction of string processing.

Q25: What is the function of the Direction Flag (DF)?

Answer: The **Direction Flag** determines the direction of string operations:

- $DF = 0 \rightarrow$ Auto-increment (processes strings from lower to higher addresses).
- $DF = 1 \rightarrow$ Auto-decrement (processes strings from higher to lower addresses).

Q26: What is the role of the Interrupt Flag (IF)?

Answer: The **Interrupt Flag** controls the recognition of external maskable interrupts.

- $IF = 1 \rightarrow$ Enables maskable interrupts.
- $IF = 0 \rightarrow$ Disables maskable interrupts.

Q27: What is the function of the Trap Flag (TF)?

Answer: The **Trap Flag** enables single-step execution mode for debugging. When $TF = 1$, the processor generates a **Trap Interrupt** after each instruction, allowing instruction-by-instruction analysis.

Q28: How are flags affected after an arithmetic operation?

Answer: Each arithmetic or logical instruction updates certain flags automatically:

- **Addition/Subtraction:** CF, ZF, SF, OF, PF, AF
- **AND/OR/XOR:** ZF, SF, PF (CF and OF cleared)

Thus, flags act as feedback indicators for the next instruction.

Q29: Why is the Flag Register important for program control?

Answer: The Flag Register enables decision-making in assembly programs by:

- Supporting conditional jumps and loops.
- Allowing signed/unsigned comparisons.
- Managing interrupt handling and debugging.

Hence, it plays a vital role in both arithmetic logic and program flow control.

Q30: What are Pointer and Index Registers in 8086?

Answer: Pointer and Index Registers are 16-bit registers used to hold **offset addresses** within segments for data manipulation, stack handling, and string processing. They help in effective addressing and data transfer operations.

Q31: Which registers belong to the Pointer and Index group?

Answer: The group includes five registers:

SP, BP, SI, DI, and IP

- SP – Stack Pointer
- BP – Base Pointer
- SI – Source Index
- DI – Destination Index
- IP – Instruction Pointer

Q32: What is the function of the Stack Pointer (SP)?

Answer: The **Stack Pointer (SP)** holds the offset address of the top of the stack in the Stack Segment (SS).

$$\text{Stack Address} = (SS \times 10H) + SP$$

It is automatically updated during PUSH and POP operations.

Q33: How does the Stack grow in 8086?

Answer: In the 8086 microprocessor, the stack grows **toward lower memory addresses**. When data is pushed, SP **decreases**; when popped, SP **increases**.

Q34: What is the purpose of the Base Pointer (BP)?

Answer: The **Base Pointer (BP)** is used to access data and parameters stored in the stack. It is mainly used in procedures and stack-based addressing:

$$\text{Address} = (SS \times 10H) + BP$$

Example: Accessing function parameters using [BP+4].

Q35: What is the role of the Source Index (SI) register?

Answer: The **Source Index (SI)** acts as a pointer to the **source string** in the Data Segment (DS).

$$\text{Source Address} = (DS \times 10H) + SI$$

It is used in string instructions like MOVSB, CMPSB, etc.

Q36: How does the Direction Flag (DF) affect SI and DI?

Answer:

- If $DF = 0 \rightarrow$ SI and DI are incremented after each string operation.
- If $DF = 1 \rightarrow$ SI and DI are decremented after each string operation.

This controls the direction of string processing.

Q37: What is the function of the Destination Index (DI)?

Answer: The **Destination Index (DI)** points to the destination string in the Extra Segment (ES).

$$\text{Destination Address} = (ES \times 10H) + DI$$

It works with SI for operations like MOVSB, STOSB, and SCASB.

Q38: What is the Instruction Pointer (IP) and its role?

Answer: The **Instruction Pointer (IP)** holds the offset of the next instruction in the Code Segment (CS).

$$\text{Code Address} = (CS \times 10H) + IP$$

It automatically updates as instructions are fetched and executed.

Q39: Can the Instruction Pointer (IP) be directly modified?

Answer: No, the IP register **cannot be directly modified** using the MOV instruction. It is changed indirectly by control transfer instructions such as JMP, CALL, and RET.

Q40: How do pointer and index registers interact with memory segments?

Answer: Each register works with a specific segment:

- SP, BP → Stack Segment (SS)
- SI → Data Segment (DS)
- DI → Extra Segment (ES)
- IP → Code Segment (CS)

Together, they generate 20-bit physical addresses for memory access.

Q41: Why are these registers essential for 8086 operation?

Answer: They support key CPU operations:

- Stack management and procedure calls (SP, BP)
- String processing and data transfer (SI, DI)
- Instruction sequencing and flow control (IP)

Thus, they form the backbone of efficient addressing and control in the 8086 microprocessor.

Q42: Calculate Physical Address using Stack Pointer (SP)

Given:

$$SS = 2500H, \quad SP = 1234H$$

Find the physical address of the top of the stack.

Solution:

$$\begin{aligned} \text{Physical Address} &= (SS \times 10H) + SP \\ &= (2500H \times 10H) + 1234H = 25000H + 1234H = \boxed{26234H} \end{aligned}$$

Q43: Compute Effective Address using BP + Displacement

Given:

$$SS = 3000H, \quad BP = 0020H, \quad \text{Displacement} = 0010H$$

Find the physical address accessed by `MOV AX, [BP + 10H]`.

Solution:

$$\text{Effective Address} = BP + \text{Displacement} = 0020H + 0010H = 0030H$$

$$\text{Physical Address} = (SS \times 10H) + 0030H = 30000H + 0030H = \boxed{30030H}$$

Q44: String Operation – Source and Destination Index

Given:

$$DS = 1000H, \quad ES = 2000H, \quad SI = 0500H, \quad DI = 0700H$$

What are the physical addresses of source and destination bytes during a `MOVSB` instruction?

Solution:

$$\text{Source Address} = (DS \times 10H) + SI = 10000H + 0500H = \boxed{10500H}$$

$$\text{Destination Address} = (ES \times 10H) + DI = 20000H + 0700H = \boxed{20700H}$$

After `MOVSB`, both SI and DI increment (if `DF = 0`):

$$SI = 0501H, \quad DI = 0701H$$

Q45: Instruction Pointer (IP) Calculation

Given:

$$CS = 2A00H, \quad IP = 0100H$$

Find the physical address of the instruction fetched next.

Solution:

$$\text{Physical Address} = (CS \times 10H) + IP = 2A000H + 0100H = \boxed{2A100H}$$

Q46: Stack Operation Effect on SP

Initially, $SP = 2000H$. After two `PUSH` operations, what is the value of SP?

Solution: Each `PUSH` decreases SP by 2 bytes:

$$SP_{\text{final}} = 2000H - 2H - 2H = \boxed{1FFCH}$$

Check: After two `POP` operations, SP returns to $2000H$.

Q47: Procedure Call and Return (BP Usage Example)

During a procedure call:

$$SP = 1000H, \quad BP = SP$$

Two local variables and one parameter are stored:

$$[BP - 2] = LocalVar1, \quad [BP - 4] = LocalVar2, \quad [BP + 4] = Parameter1$$

Question: Which address is accessed by `MOV AX, [BP+4]` if $SS = 3000H$?

Solution:

$$\text{Effective Address} = BP + 4 = 1000H + 4H = 1004H$$

$$\text{Physical Address} = (SS \times 10H) + 1004H = 30000H + 1004H = \boxed{31004H}$$

Q48: Effective Address using BX + SI + Displacement

Given:

$$DS = 4000H, \quad BX = 0200H, \quad SI = 0100H, \quad \text{Displacement} = 0020H$$

Find the physical address accessed by `MOV AX, [BX + SI + 20H]`.

Solution:

$$\text{Effective Address} = BX + SI + \text{Displacement} = 0200H + 0100H + 0020H = 0320H$$

$$\text{Physical Address} = (DS \times 10H) + 0320H = 40000H + 0320H = \boxed{40320H}$$

Q49: PUSH and POP Operations on Stack

Initially:

$$SP = 3000H$$

After executing:

$$\text{PUSH AX}, \quad \text{PUSH BX}, \quad \text{POP CX}$$

Find the final value of SP.

Solution: Each PUSH decreases SP by 2 bytes, each POP increases SP by 2 bytes:

$$SP_{\text{after PUSH AX}} = 3000H - 2H = 2FFE H$$

$$SP_{\text{after PUSH BX}} = 2FFE H - 2H = 2FFCH$$

$$SP_{\text{after POP CX}} = 2FFCH + 2H = 2FFE H$$

$$\text{Final SP} = \boxed{2FFE H}$$

Q50: Physical Address of Instruction using CS and IP

Given:

$$CS = 1B00H, \quad IP = 0F00H$$

Find the physical address of the next instruction fetch.

Solution:

$$\text{Physical Address} = (CS \times 10H) + IP = 1B000H + 0F00H = \boxed{1BF00H}$$

Q51: MOV Immediate to Memory with Displacement

Given:

$$DS = 5000H, \quad BX = 0100H, \quad \text{Displacement} = 0020H$$

Instruction: MOV [BX + 20H], 1234H

Solution:

$$\text{Effective Address} = BX + \text{Displacement} = 0100H + 0020H = 0120H$$

$$\text{Physical Address} = (DS \times 10H) + 0120H = 50000H + 0120H = \boxed{50120H}$$

Instruction Size: 4 bytes (Opcode + ModR/M + 16-bit immediate)

Q52: Direction Flag Effect in String Operation

Given:

$$SI = 0100H, \quad DI = 0200H, \quad DF = 1$$

Instruction: MOVSW

Solution:

- Source Address: $DS:SI = DS * 10H + SI = (\text{assume } DS=1000H) \rightarrow 10000H + 0100H = 10100H$
- Destination Address: $ES:DI = ES * 10H + DI = (\text{assume } ES=2000H) \rightarrow 20000H + 0200H = 20200H$
- Since $DF=1$, SI and DI decrement by 2 after operation:

$$SI = 0100H - 2H = 00FEH, \quad DI = 0200H - 2H = 01FEH$$

Q53: CALL and RET Instruction Stack Behavior

Initially:

$$SP = 4000H, \quad CS = 3000H, \quad IP = 0100H$$

Instruction: CALL 2000H

Solution:

- CALL pushes current IP onto stack: $SP = 4000H - 2H = 3FFE H$
- IP of next instruction is 2000H (jump destination)
- Physical address of return address stored on stack: $SS * 10H + SP = (\text{assume } SS=3000H) \rightarrow 30000H + 3FFE H = 33FFE H$

Q54: LEA Instruction – Load Effective Address

Instruction: LEA AX, [BX + SI + 10H] Given:

$$BX = 0100H, \quad SI = 0200H$$

Solution:

$$\text{Effective Address} = BX + SI + 10H = 0100H + 0200H + 0010H = 0310H$$

$$AX \leftarrow 0310H$$

Q55: Stack Pointer after Multiple PUSH Operations

Initially: $SP = 3000H$ Operations: PUSH AX, PUSH BX, PUSH CX, PUSH DX

Solution: Each PUSH decreases SP by 2:

$$SP = 3000H - 2H - 2H - 2H - 2H = 2FF8H$$

Q56: Compute Effective Address with BX + SI + Displacement

Given:

$$DS = 3000H, \quad BX = 0100H, \quad SI = 0200H, \quad \text{Displacement} = 0010H$$

Find the physical address accessed by MOV AX, [BX + SI + 10H].

Solution:

$$\text{Effective Address} = BX + SI + \text{Displacement} = 0100H + 0200H + 0010H = 0310H$$

$$\text{Physical Address} = (DS \times 10H) + 0310H = 30000H + 0310H = \boxed{30310H}$$

Q57: Determine Instruction Length

Instruction: `MOV AX, [BP+DI+1234H]`

Question: How many bytes does this instruction occupy?

Solution:

- Opcode byte = 1 byte
- MOD-REG-R/M byte = 1 byte
- 16-bit displacement = 2 bytes
- Total instruction size = $1 + 1 + 2 =$ 4 bytes

Q58: Identify Opcode Byte Fields

Instruction: `MOV AX, [BX]` **Question:** What are the Opcode, D, and W bits for this instruction?

Solution:

- Opcode (6 bits) = 100010 (`MOV r/m16, r16`)
- D bit = 1 (destination = AX)
- W bit = 1 (16-bit operation)
- First byte encoding = 10001011b

Q59: MOD-REG-R/M Byte Calculation

Instruction: `MOV AX, [BX+SI+05H]`

Solution:

MOD = 01 (8-bit displacement), REG = 000 (AX), R/M = 000 (BX+SI)

MOD-REG-R/M byte = 01000000b

Effective Address = $BX + SI + 05H$

Q60: Immediate Data Encoding

Instruction: `MOV AX, 1234H`

Question: Encode the instruction in bytes.

Solution:

- Opcode byte: B8H (10110000b) → `MOV immediate to AX, W=1`
- Immediate data: 34H (low byte), 12H (high byte)
- Total instruction bytes: B8 34 12
- Instruction length: 3 bytes

Q61: Determine MOD Field from Addressing

Instruction: `MOV AX, [BP+DI+1234H]`

Question: Which MOD value is used and why?

Solution:

- $\text{MOD} = 10 \rightarrow 16\text{-bit displacement}$
- Reason: BP + DI used with 16-bit displacement (1234H)

Q62: Effective Address Calculation

Instruction: `MOV AX, [SI+1234H]` Given: DS = 2000H, SI = 0100H

Solution:

$$\text{Effective Address} = SI + \text{Displacement} = 0100H + 1234H = 1334H$$

$$\text{Physical Address} = DS \times 10H + 1334H = 20000H + 1334H = \boxed{21334H}$$

Q63: Instruction Size Calculation

Instruction: `MOV [1234H], AX`

Solution:

- Opcode byte = 1
- MOD-REG-R/M byte = 1
- 16-bit memory address = 2
- Total instruction size = $1 + 1 + 2 = \boxed{4 \text{ bytes}}$

Q64: Direct Memory-to-Memory Restriction

Question: Is the following instruction allowed in 8086? `MOV [DI], [SI]`

Solution:

- **Not allowed.** 8086 cannot perform memory-to-memory transfer directly.
- Workaround: Use a register as intermediary:

```
MOV AH, [SI]
MOV [DI], AH
```

Q65: Little-Endian Immediate Data

Instruction: MOV AX, 1234H

Question: How is the immediate data stored in memory?

Solution:

- Little-endian format: low byte first, high byte next
- 1234H → 34H (low byte), 12H (high byte)
- Memory storage: 34H at lower address, 12H at higher address

Q66: MOD-REG-R/M Byte for Register-to-Register

Instruction: MOV BX, CX

Solution:

- MOD = 11 (register-to-register)
- REG = 001 (CX), R/M = 011 (BX)
- D = 1 (destination = BX), W = 1 (16-bit)
- MOD-REG-R/M byte = 11 001 011 = 11001011b
- Instruction length: 2 bytes

Q67: Effective Address Calculation with BP+SI

Given: SS = 2000H, BP = 0100H, SI = 0200H

Instruction: MOV AX, [BP+SI]

Solution:

$$EA = BP + SI = 0100H + 0200H = 0300H$$

$$\text{Physical Address} = SS \times 10H + 0300H = 20000H + 0300H = \boxed{20300H}$$

Q68: D and W Bit Usage

Instruction: MOV [BX], AL

Solution:

- Destination = memory → D = 0 (source = AL)
- AL = 8-bit → W = 0
- Opcode byte = 10001000b

Q69: 8-bit Displacement Example

Instruction: MOV AX, [BX+05H]

Solution:

- MOD = 01 → 8-bit displacement
- EA = BX + 05H
- MOD-REG-R/M byte = 01 (MOD) + 000 (REG=AX) + 000 (R/M=BX) = 01000000b
- Instruction length = 3 bytes (Opcode + MOD-REG-R/M + 8-bit displacement)

Q70: 16-bit Displacement Example

Instruction: MOV AX, [BP+DI+1234H]

Solution:

- MOD = 10 → 16-bit displacement
- EA = BP + DI + 1234H
- Instruction length = 4 bytes (Opcode + MOD-REG-R/M + 16-bit displacement)

Q71: Instruction Length – Immediate to Register

Instruction: MOV CX, 5678H

Solution:

- Opcode byte = B9H (MOV immediate to CX)
- Immediate = 16-bit → 78H 56H (little-endian)
- Total instruction length = 1 + 2 = 3 bytes

Q72: Direct Memory Address Example

Instruction: MOV AX, [2000H] **Given:** DS = 1000H

Solution:

$$\text{Physical Address} = DS \times 10H + 2000H = 10000H + 2000H = \boxed{12000H}$$

Instruction length = 1 (Opcode) + 1 (MOD-REG-R/M) + 2 (16-bit address) = 4 bytes

Q73: MOD Field Exception

Instruction: MOV AX, [BP]

Solution:

- MOD = 01 (cannot use MOD=00 with BP alone)
- 8-bit displacement = 00H
- EA = BP + 00H

Q74: Source and Destination in Register-to-Memory

Instruction: MOV [BX+SI], DX

Solution:

- Memory operand = destination $\rightarrow D = 0$
- Word operation $\rightarrow W = 1$
- MOD = 00 (no displacement)
- REG = 010 (DX), R/M = 000 (BX+SI)
- MOD-REG-R/M byte = 00010000b

Q75: Effective Address with SI + Displacement

Instruction: MOV AX, [SI+12H] **Given:** DS = 3000H, SI = 0100H

Solution:

$$EA = SI + 12H = 0100H + 0012H = 0112H$$

$$\text{Physical Address} = DS \times 10H + 0112H = 30000H + 0112H = \boxed{30112H}$$

Q76: MOD-REG-R/M for BP+DI+Displacement

Instruction: MOV AX, [BP+DI+0A0AH]

Solution:

- MOD = 10 $\rightarrow$ 16-bit displacement
- REG = 000 (AX), R/M = 011 (BP+DI)
- MOD-REG-R/M byte = 10000011b
- Instruction length = 1 (opcode) + 1 (MOD-REG-R/M) + 2 (displacement) = 4 bytes

Q77: Immediate-to-Register Example

Instruction: MOV AL, 5AH

Solution:

- $W = 0 \rightarrow$ 8-bit operation
- Opcode byte = B0H (MOV immediate to AL)
- Immediate data = 5AH
- Total length = 2 bytes (Opcode + immediate)

Q78: Instruction Length – Register-to-Register Word Transfer

Instruction: MOV DX, BX

Solution:

- Both operands are registers $\rightarrow$ MOD = 11
- Opcode byte = 100010b + D=1, W=1 $\rightarrow$ 10001011b
- MOD-REG-R/M byte = REG = BX (011), R/M = DX (010) $\rightarrow$ 11011010b
- Instruction length = 2 bytes

Q79: Displacement Calculation with BX+DI

Instruction: MOV AX, [BX+DI+1F2H]

Solution:

$$EA = BX + DI + 1F2H$$

MOD = 10 (16-bit displacement), instruction length = 4 bytes

Q80: Physical Address of MOV with BP+SI

Given: SS = 4000H, BP = 0200H, SI = 0300H Instruction: MOV AX, [BP+SI]

Solution:

$$EA = BP + SI = 0200H + 0300H = 0500H$$

$$\text{Physical Address} = SS \times 10H + 0500H = 40000H + 0500H = \boxed{40500H}$$

Q81: Maximum Addressable Memory

Question: What is the maximum memory space that the 8086 microprocessor can address?

Solution: 8086 has a 20-bit address bus.

$$\text{Maximum Memory} = 2^{20} = 1 \text{ MB}$$

Memory range: 000000H to FFFFFFFH.

Q82: Segment Size

Question: What is the size of each memory segment in 8086?

Solution: Each segment = 2^{16} bytes = 64 kB.

Q83: Physical Address Calculation

Question: Calculate the physical address for logical address DS:1234H if DS = 1000H.

Solution:

$$\begin{aligned}\text{Physical Address} &= (DS \times 10H) + \text{Offset} \\ &= (1000H \times 10H) + 1234H = 10000H + 1234H = 11234H\end{aligned}$$

Q84: Base Address of a Segment

Question: Find the base address of segment 2000H.

Solution:

$$\text{Base Address} = \text{Segment} \times 10H = 2000H \times 10H = 20000H$$

Q85: Offset Range

Question: What is the offset range in 8086 memory segmentation?

Solution: Offset is a 16-bit value, so:

$$\text{Offset Range} = 0000H \text{ to } FFFFH$$

Q86: Active Segments

Question: How many memory segments can be active at any time in 8086?

Solution: At most 4 segments: CS, DS, SS, ES.

Q87: Logical Address Format

Question: Represent a logical address in 8086.

Solution: Logical Address = Segment : Offset Example: DS:1234H

Q88: Interrupt Vector Table

Question: Where is the Interrupt Vector Table (IVT) located? How many bytes does it occupy?

Solution: IVT: 00000H to 003FFH (1 KB). Contains 256 entries, 4 bytes each.

Q89: Reset Vector

Question: After reset, from which physical address does 8086 start execution?

Solution: Physical Address = FFFF0H Segment = FFFFH, Offset = 0000H

Q90: Maximum Active Memory

Question: Explain why only 256 kB memory is “active” at a time.

Solution: Only 4 segment registers can be referenced at a time. Each segment = 64 kB $\rightarrow 4 \times 64 \text{ kB} = 256 \text{ kB}$ accessible directly. Other segments require reloading segment registers.

Q91: Overlapping Segments

Question: Explain segment overlapping with an example.

Solution: Segments start at multiples of 16 bytes, so physical addresses may overlap. Example: - Segment1 = 1000H $\rightarrow$ 10000H–1FFFFH - Segment2 = 1001H $\rightarrow$ 10010H–1FFFFH Overlap: 10010H onward.

Q92: Memory Banks in 8086

Question: How is memory organized into banks for 16-bit access?

Solution: Two banks: - Low Bank (even addresses) $\rightarrow$ D0–D7 - High Bank (odd addresses) $\rightarrow$ D8–D15

Q93: Bank Selection using A0 and BHE

Question: How do A0 and BHE select banks?

Solution:	BHE	A0	Access Type
	0	0	Both banks $\rightarrow$ 16-bit word
	0	1	High bank $\rightarrow$ odd byte
	1	0	Low bank $\rightarrow$ even byte
	1	1	No valid access

Q94: Odd-Addressed Word Access

Question: Why does accessing a 16-bit word at an odd address require two memory cycles?

Solution: Word straddles two banks: 1. First cycle: high byte from odd address (D8–D15) 2. Second cycle: low byte from next even address (D0–D7)

Q95: Effective Address Calculation

Question: Calculate the physical address for logical address SS:1234H if SS = 2000H.

Solution:

$$\text{Physical Address} = (SS \times 10H) + 1234H = 20000H + 1234H = 21234H$$

Q96: Physical Address Calculation

Question: Calculate the physical address for logical address CS:5678H if CS = 2000H.

Solution:

$$\text{Physical Address} = (CS \times 10H) + 5678H = 20000H + 5678H = 25678H$$

Q97: Logical Address Format

Question: Represent the logical address for data stored in DS segment with offset 1A3FH.

Solution:

$$\text{Logical Address} = DS : 1A3FH$$

Q98: BDA Location

Question: Specify the BIOS Data Area (BDA) memory range.

Solution:

Location: 00400H – 004FFH (256 bytes)

Q99: Even-Addressed Byte

Question: How is an even-addressed byte accessed?

Solution:

Condition: A0 = 0, BHE = 1 → Low bank (D0–D7) enabled

Q100: Odd-Addressed Byte

Question: How is an odd-addressed byte accessed?

Solution:

Condition: A0 = 1, BHE = 0 → High bank (D8–D15) enabled

Q101: Even-Addressed Word

Question: Explain accessing a 16-bit word at an even address.

Solution:

Condition: A0 = 0, BHE = 0 → Both banks enabled.

Low byte → D0–D7, High byte → D8–D15, single memory cycle

Q102: Odd-Addressed Word

Question: Explain accessing a 16-bit word at an odd address.

Solution:

Requires 2 memory cycles:

1. High byte from odd address → D8–D15
2. Low byte from next even address → D0–D7

Q103: Immediate Addressing Mode

Question: What is Immediate Addressing Mode?

Solution:

Operand is specified explicitly in the instruction. Example: `MOV AX, 1234H` → 1234H is the immediate data

Q104: Register Addressing Mode

Question: What is Register Addressing Mode?

Solution:

Operand is located in a register. Example: `MOV AX, BX` → Contents of BX moved to AX

Q105: Direct Addressing Mode

Question: Explain Direct Addressing Mode.

Solution:

Memory address of the operand is specified in the instruction. Example: `MOV AX, [1234H]` → Data at memory location 1234H moved to AX

Q106: Register Indirect Addressing Mode

Question: Explain Register Indirect Addressing Mode.

Solution:

Memory address is held in a register. Example: `MOV AX, [BX]` → Contents of memory at address in BX moved to AX

Q107: Based Addressing Mode

Question: What is Based Addressing Mode?

Solution:

Effective address = Base register + displacement. Example: `MOV AX, [BX + 05H]` → Data at (BX + 05H) moved to AX

Q108: Indexed Addressing Mode

Question: What is Indexed Addressing Mode?

Solution:

Effective address = Index register + displacement. Example: `MOV AX, [SI + 02H]` → Data at (SI + 02H) moved to AX

Q109: Based-Indexed Addressing Mode

Question: Explain Based-Indexed Addressing Mode.

Solution:

Effective address = Base register + Index register + optional displacement. Example: `MOV AX, [BX + SI + 04H]` → Data at (BX + SI + 04H) moved to AX

Q110: Relative Addressing Mode

Question: What is Relative Addressing Mode?

Solution:

Operand address is determined relative to instruction pointer (IP). Example: `JMP LABEL` → Control jumps to LABEL relative to current IP

Q111: Immediate Addressing Mode Example

Question: Move immediate value 5AH into AX and calculate physical address.

Solution:

Instruction: `MOV AX, 5AH`

Explanation: Operand is directly in instruction; no memory address is accessed.

Physical address: Not applicable (immediate data in instruction)

Q112: Register Addressing Mode Example

Question: Move contents of BX to CX using register addressing.

Solution:

Instruction: `MOV CX, BX`

Explanation: Operand is in register; no memory involved. Physical address: Not applicable

Q113: Direct Addressing Mode Example

Question: Load AX from memory location 2000H in DS segment.

Solution:

Instruction: `MOV AX, [2000H]`

Physical address = $DS \times 16 + \text{Offset} = 1000H \times 16 + 2000H = 12000H$

Q114: Register Indirect Addressing Mode Example

Question: Move data pointed by BX (value = 1000H) into AX.

Solution:

Instruction: `MOV AX, [BX]`

Assuming $DS = 2000H$, Physical address = $DS \times 16 + BX = 2000H \times 16 + 1000H = 21000H$

Q115: Based Addressing Mode Example

Question: Access memory using BP as base with 06H displacement.

Solution:

Instruction: `MOV AX, [BP + 06H]`

Assume $SS = 3000H$, $BP = 100H$

Physical address = $SS \times 16 + BP + \text{Displacement} = 3000H \times 16 + 100H + 06H = 30106H$

Q116: Indexed Addressing Mode Example

Question: Move data at offset SI + 08H into AX, DS = 2000H, SI = 0100H.

Solution:

Instruction: MOV AX, [SI + 08H]

Physical address = $DS \times 16 + SI + \text{Displacement} = 2000H \times 16 + 0100H + 08H = 20108H$

Q117: Based + Indexed Addressing Mode Example

Question: Access memory using BX = 1000H, SI = 0200H, displacement = 04H, DS = 3000H.

Solution:

Instruction: MOV AX, [BX + SI + 04H]

Physical address = $DS \times 16 + BX + SI + 04H = 3000H \times 16 + 1000H + 0200H + 04H = 31004H$

Q118: Relative Addressing Mode Example

Question: Write a short jump to label LOOP with displacement = FEH, IP = 1000H.

Solution:

Instruction: JMP SHORT LOOP

Physical address calculation: Effective IP = $1000H + FEH$ (signed displacement interpreted as -02H) = 0FFE H

Q119: Register Indirect with Displacement Example

Question: Load AX from memory address BX + 0AH, BX = 200H, DS = 1000H.

Solution:

Instruction: MOV AX, [BX + 0AH]

Physical address = $DS \times 16 + BX + \text{Displacement} = 1000H \times 16 + 200H + 0AH = 1020AH$

Q120: Using BP in Based Addressing with SS Segment

Question: Move AX from stack at BP + 08H, SS = 4000H, BP = 0100H.

Solution:

Instruction: MOV AX, [BP + 08H]

Physical address = $SS \times 16 + BP + \text{Displacement} = 4000H \times 16 + 0100H + 08H = 40108H$

Q121: Combining DS and SI Example

Question: Move AX from DS:SI + 04H, DS = 3000H, SI = 0100H.

Solution:

Instruction: MOV AX, [SI + 04H]

Physical address = $DS \times 16 + SI + 04H = 3000H \times 16 + 0100H + 04H = 30104H$

Q122: Using ES Segment Example

Question: Move a word from memory at ES:DI + 06H into AX. ES = 5000H, DI = 0200H.

Solution:

Instruction: `MOV AX, ES:[DI + 06H]`

Physical address = $ES \times 16 + DI + \text{Displacement} = 5000H \times 16 + 0200H + 06H = 50206H$